

# LESSON

2

## Meet the Motors

*From Java names to real FTC hardware*

**MISSION** Can you prove that each Java motor name reaches the intended physical wheel?

### Learning targets

| CHECK | BY THE END OF THIS LESSON, I CAN...         |
|-------|---------------------------------------------|
| [ ]   | Separate declaration from hardware mapping. |
| [ ]   | Identify both hardwareMap.get() arguments.  |
| [ ]   | Audit exact configuration names.            |
| [ ]   | Perform a safe, low-power direction test.   |

### Words to know

**VOCABULARY** declaration | field | DcMotor | hardwareMap | configuration name | case-sensitive | direction

### Code focus

```
private DcMotor leftMotor;
private DcMotor rightMotor;

leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
rightMotor = hardwareMap.get(DcMotor.class, "right_motor");
```

### Before the video: commit to a prediction

**MY PREDICTION** If the configuration says left\_motor but the code says leftMotor, predict what will happen during INIT.

---



---

Confidence before the lesson: 1 2 3 4 5 (circle one)

## The POWERING ON safety contract

**NON-NEGOTIABLE** The sequence is completed before every live-motor test. If a step is skipped, remain disabled and restart at Step 1.

| STEP | ACTION                                                                                                                 | TEAM CHECK |
|------|------------------------------------------------------------------------------------------------------------------------|------------|
| 1    | SECURE THE ROBOT<br>Use stable blocks. Keep driven wheels and moving mechanisms clear of the table and floor.          | [ ]        |
| 2    | CONFIRM DISABLED<br>Select the correct robot and OpMode, but do not press PLAY.                                        | [ ]        |
| 3    | LOOK AROUND<br>Remove hands, tools, wires, scraps, and loose objects from moving areas.                                | [ ]        |
| 4    | MAKE SURE ALL ARE CLEAR<br>Everyone steps outside the boundary; secure hair, clothing, lanyards, strings, and jewelry. | [ ]        |
| 5    | CALL AND WAIT<br>Call POWERING ON! loudly. Pause long enough for everyone to stop, look up, and step away.             | [ ]        |
| 6    | VISUALLY CONFIRM, THEN ENABLE<br>Scan the full area again. Press PLAY only when everyone and everything is clear.      | [ ]        |

### Callout script

**BEFORE PLAY** Robot is secured and DISABLED. Look around. Everyone clear? POWERING ON! ... Wait ... Area is visibly clear. Enabling now.

**BEFORE CONTACT** STOP pressed. Driver Station shows DISABLED. Hands may approach only after the operator confirms disabled.

### Student agreement

I understand that anyone may call STOP and the operator will disable immediately. I will not approach or touch the robot until DISABLED is visually confirmed.

Student signature: \_\_\_\_\_ Date: \_\_\_\_\_

## Reference program: BasicTwoMotorTeleOp.java

Use this numbered reference throughout Lessons 1-6. The comments divide the program into startup and repeating control sections.

```

01 package org.firstinspires.ftc.teamcode;
02
03 import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
04 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
05 import com.qualcomm.robotcore.hardware.DcMotor;
06
07 @TeleOp(name="Basic Two Motor TeleOp", group="Iterative Opmode")
08 public class BasicTwoMotorTeleOp extends LinearOpMode {
09     private DcMotor leftMotor;
10     private DcMotor rightMotor;
11
12     @Override
13     public void runOpMode() {
14         leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
15         rightMotor = hardwareMap.get(DcMotor.class, "right_motor");
16
17         // Set one motor REVERSE if the physical drivetrain requires it.
18         // rightMotor.setDirection(DcMotor.Direction.REVERSE);
19
20         telemetry.addData("Status", "Initialized");
21         telemetry.update();
22         waitForStart();
23
24         if (opModeIsActive()) {
25             while (opModeIsActive()) {
26                 double leftPower = -gamepad1.left_stick_y;
27                 double rightPower = -gamepad1.right_stick_y;
28                 leftMotor.setPower(leftPower);
29                 rightMotor.setPower(rightPower);
30                 telemetry.addData("Left Power", leftPower);
31                 telemetry.addData("Right Power", rightPower);
32                 telemetry.update();
33             }
34         }
35     }
36 }

```

**CODE DETECTIVE** Circle `waitForStart()`. Box the while condition. Underline both `setPower()` calls. Draw a star beside each telemetry line.

**LESSON****2****Guided Video Notes***Meet the Motors*

**DIRECTIONS** At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

| PAUSE | QUESTION                                                 | MY CODE / VISUAL EVIDENCE |
|-------|----------------------------------------------------------|---------------------------|
| 0:25  | What does declaring a variable accomplish?               |                           |
| 0:34  | Does declaring leftMotor connect it to a physical motor? |                           |
| 0:45  | What job does hardwareMap perform?                       |                           |
| 0:56  | What are the two hardwareMap.get() arguments?            |                           |
| 1:06  | Why must left_motor include the underscore?              |                           |
| 1:23  | Why might one drivetrain motor need reversal?            |                           |
| 1:42  | What is the safe first-direction-test sequence?          |                           |

**MOST IMPORTANT IDEA** The idea I would teach to a teammate is:

---



---

**LESSON****2****Hands-On Team Activity***Meet the Motors***Materials and setup**

| READY | ITEM                                                                                |
|-------|-------------------------------------------------------------------------------------|
| [ ]   | Robot secured on stable blocks with both drive wheels fully clear.                  |
| [ ]   | Robot Controller configuration and BasicTwoMotorTeleOp.java displayed side by side. |
| [ ]   | Printed mapping and direction-observation sheet from this guide.                    |
| [ ]   | Gamepad and Driver Station; POWERING ON safety poster visible at the test boundary. |
| [ ]   | Optional printed mismatch cards: left_motor, leftMotor, Left_Motor, and left motor. |

**Team roles**

| ROLE             | JOB                                                      |
|------------------|----------------------------------------------------------|
| Safety lead      | Reads each POWERING ON step and watches the boundary.    |
| Driver Station   | Selects INIT, PLAY, and STOP only when directed.         |
| Code reader      | Finds declarations and hardwareMap.get() lines.          |
| Config navigator | Reads the configured device names exactly.               |
| Left observer    | Records left-wheel direction from outside the boundary.  |
| Right observer   | Records right-wheel direction from outside the boundary. |
| Recorder         | Completes the mapping and direction tables.              |
| Debugger         | Proposes one evidence-based correction after STOP.       |

**Activity sequence**

| STEP | ACTION                                                                                                                                                                |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Point to <code>private DcMotor leftMotor;</code> . Say: This creates a typed Java name. Has Java connected to the real motor yet?                                     |
| 2    | Read each <code>hardwareMap.get()</code> call from the inside out: device type, quoted configuration name, returned motor, assigned variable.                         |
| 3    | Show one incorrect name card. Ask students to predict when the failure occurs and whether PLAY should be pressed.                                                     |
| 4    | Robot secured. Confirm DISABLED. Look around. Make sure everyone is clear. Call POWERING ON! Wait. Visually confirm clear. Then direct the operator to INIT and PLAY. |
| 5    | After STOP and DISABLED, compare commanded side, observed side, configuration name, and direction setting. Change only one cause before retesting.                    |

# LESSON 2

## Lab Record and Exit Ticket

*Meet the Motors*

Student/team: \_\_\_\_\_ Date: \_\_\_\_\_ Robot: \_\_\_\_\_

| BEFORE PLAY                                                                                                                                                                                                                              | AFTER TEST                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> Robot secure <input type="checkbox"/> DISABLED <input type="checkbox"/> Area clear<br><input type="checkbox"/> POWERING ON called <input type="checkbox"/> Wait completed <input type="checkbox"/> Visual clear | <input type="checkbox"/> STOP pressed<br><input type="checkbox"/> DISABLED visually confirmed<br>Safety lead initials: _____ |

### Motor mapping and direction record

| CHECK                        | LEFT SIDE     | RIGHT SIDE    | EVIDENCE / NOTES    |
|------------------------------|---------------|---------------|---------------------|
| Java variable                | leftMotor     | rightMotor    |                     |
| Device type                  | DcMotor.class | DcMotor.class |                     |
| Quoted code name             |               |               | Copy exactly        |
| Robot Configuration name     |               |               | Copy exactly        |
| Exact match?                 | Yes / No      | Yes / No      | Circle differences  |
| Gentle forward-stick test    | Wheel: _____  | Wheel: _____  | Expected: _____     |
| Direction correction needed? | Yes / No      | Yes / No      | Only after DISABLED |

### Debrief

| # | QUESTION                                                | MY RESPONSE |
|---|---------------------------------------------------------|-------------|
| 1 | What is created by private DcMotor leftMotor;?          |             |
| 2 | What are the two key hardwareMap.get() arguments?       |             |
| 3 | Why do we test one side at a time?                      |             |
| 4 | What must happen before anyone approaches after a test? |             |

### Exit ticket

**ANSWER BEFORE LEAVING** Explain the difference between leftMotor and "left\_motor" in one or two sentences.

\_\_\_\_\_

\_\_\_\_\_